

EXPERIMENT NO. 1

Comparative Study of Low-Code Development Versus Traditional Application Development Approaches

Thaneshwar M

240701561

1. Aim

To conduct a comparative study of traditional application development and low-code application development approaches by examining their development stages, implementation methods, advantages, limitations, development speed, cost, flexibility, maintenance, and suitability for different types of applications.

2. Introduction to Traditional Approach

Traditional application development is a conventional software development method in which applications are designed, coded, tested, deployed, and maintained using programming languages, frameworks, databases, and development tools. It provides developers with complete control over the application's architecture, source code, user interface, business logic, and integrations.

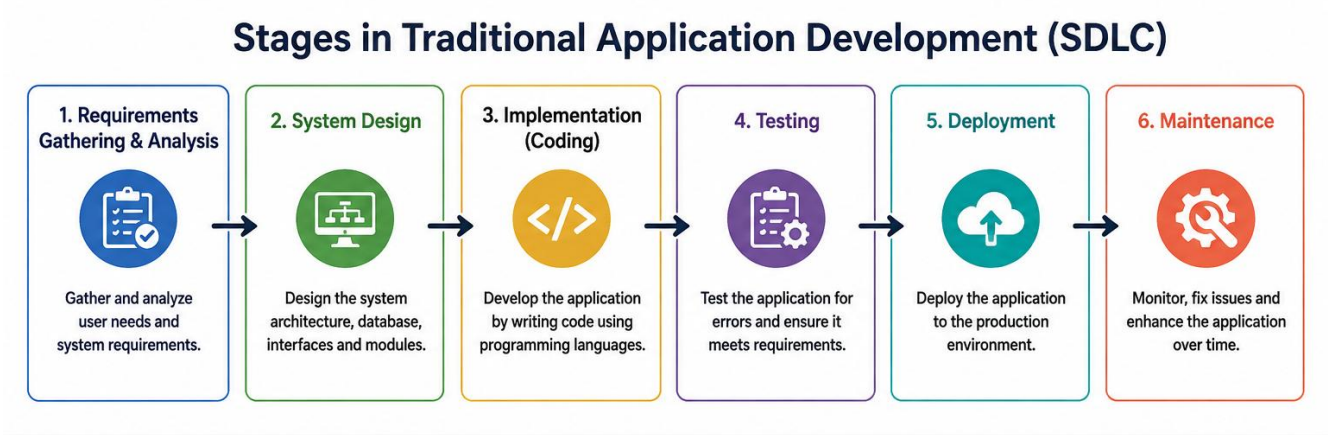
It generally follows the Software Development Life Cycle (SDLC) using methodologies such as Waterfall or Agile. This approach is suitable for complex and highly customized applications, but it often requires more development time, programming expertise, testing effort, and maintenance.

3. Introduction to Low-Code Approach

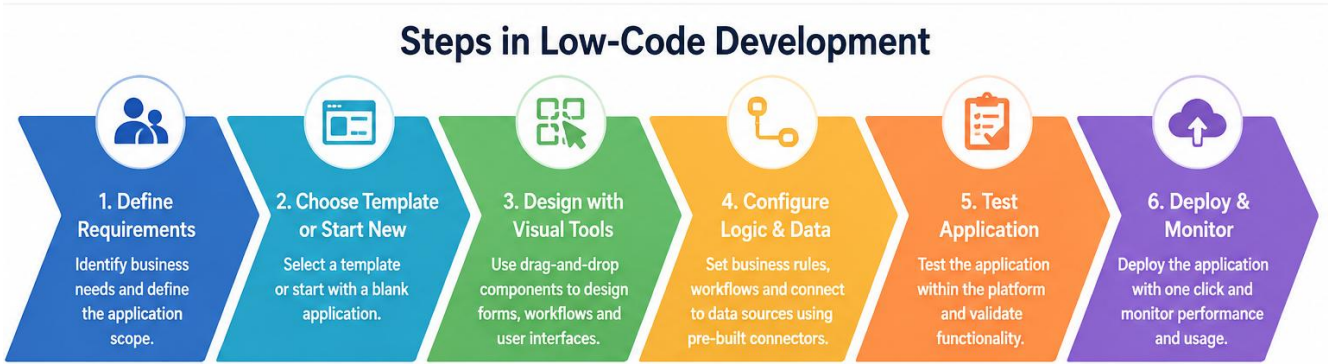
Low-code development is an application development approach that reduces the amount of manually written source code by providing visual development environments, reusable components, drag-and-drop interfaces, pre-built workflows, templates, connectors, and automated deployment capabilities. Developers and other authorized users can construct applications by configuring these components and writing limited custom code when necessary.

Low-code platforms are intended to accelerate application delivery while reducing repetitive coding activities. They can be used for business applications, workflow automation, dashboards, forms, internal tools, and other applications where rapid delivery and integration are important. Although low-code reduces manual coding, it does not eliminate the need for software engineering practices such as requirements analysis, security, testing, performance evaluation, version control, governance, and maintenance.

4. Diagram for Stages in Traditional Application Development



5. Diagram for Stages in Traditional Approach



6. Comparative Analysis (Tabulation)

Parameter	Traditional Development	Low-Code Development
Primary approach	Manual programming using languages, frameworks, libraries, and development tools.	Visual development using pre-built components, configuration, workflows, and limited custom code.

Development speed	Generally slower because more functionality is implemented manually.	Generally faster because reusable components and automation reduce repetitive coding.
Programming knowledge	Requires strong programming and software engineering skills.	Requires less coding for common tasks, but technical knowledge is still valuable for complex requirements.
Customization	Very high; developers have detailed control over implementation.	High for supported features; advanced customization may depend on platform extensibility.
User interface development	Usually coded or assembled using UI frameworks and libraries.	Often created using drag-and-drop components and visual configuration.
Database development	Schemas, queries, data access layers, and migrations are commonly designed and implemented manually.	Data models and connections can often be configured using visual tools and platform services.

7. Result

The comparative study of traditional and low-code application development approaches was completed successfully. Traditional development was found to provide greater control, flexibility, and customization through direct programming, while low-code development reduces repetitive coding effort and can significantly accelerate application delivery through visual tools, reusable components, workflows, and integrations.

Therefore, neither approach is universally superior. Traditional development is more appropriate for highly customized, complex, performance-critical, or specialized systems, whereas low-code development is particularly suitable when rapid delivery, business process automation, standard application functionality, and reduced development effort are priorities. A hybrid approach may also be adopted when a project requires both rapid low-code development and custom programming for specialized functionality.